

# Raport AI

## Raport wykorzystania sztucznej inteligencji podczas realizacji projektu

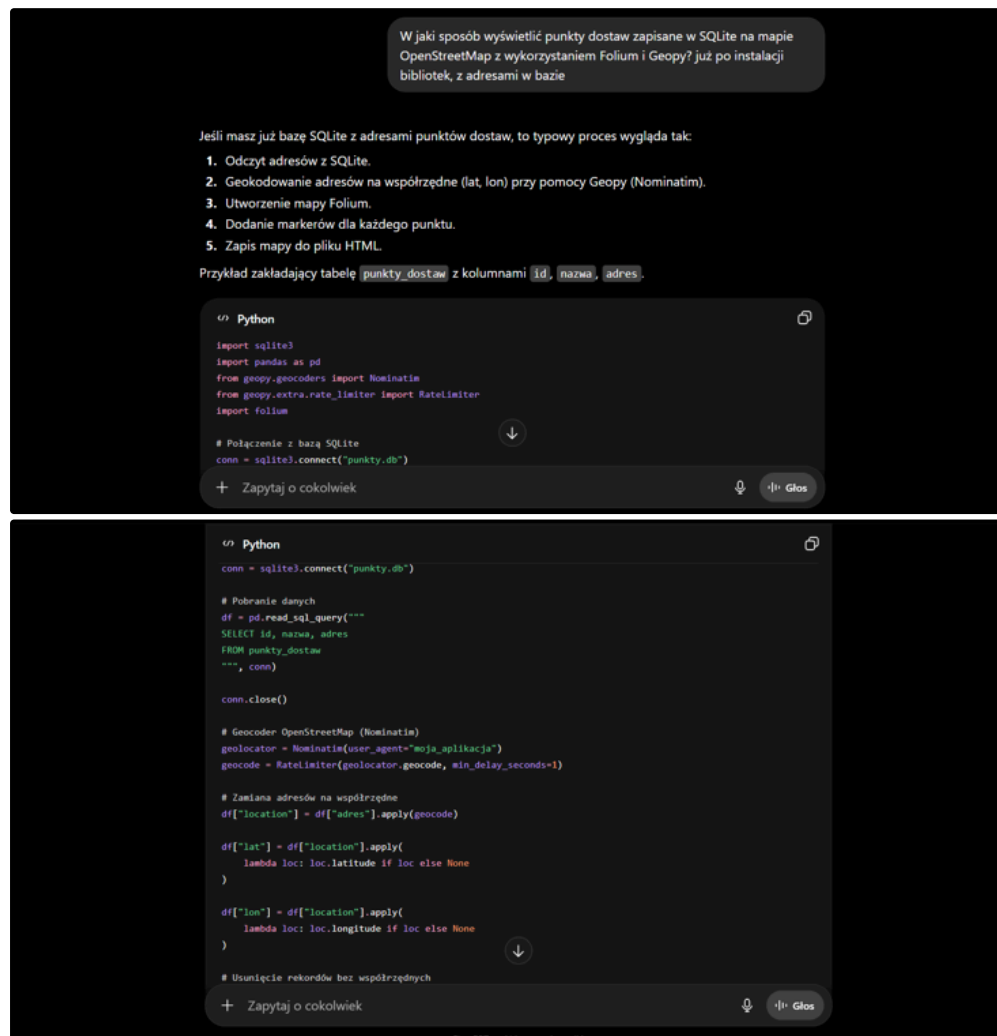
### 1. Cel wykorzystania sztucznej inteligencji

Podczas realizacji projektu „System Planowania Tras Kuriera” wykorzystano narzędzia oparte na sztucznej inteligencji jako wsparcie procesu tworzenia aplikacji. Celem wykorzystania AI(ChatGPT) było przyspieszenie pracy, uzyskanie wyjaśnień dotyczących napotkanych problemów oraz pomoc przy projektowaniu i dokumentowaniu rozwiązania.

### 2. Obszary wykorzystania AI

Sztuczna inteligencja została wykorzystana w następujących obszarach:

- wyjaśnianie działania bibliotek Python (Tkinter, SQLite, Folium, Geopy)



W jaki sposób wyświetlić punkty dostaw zapisane w SQLite na mapie OpenStreetMap z wykorzystaniem Folium i Geopy? już po instalacji bibliotek, z adresami w bazie

Jeśli masz już bazę SQLite z adresami punktów dostaw, to typowy proces wygląda tak:

1. Odczyt adresów z SQLite.
2. Geokodowanie adresów na współrzędne (lat, lon) przy pomocy Geopy (Nominatim).
3. Utworzenie mapy Folium.
4. Dodanie markerów dla każdego punktu.
5. Zapis mapy do pliku HTML.

Przykład zakładający tabelę `punkty_dostaw` z kolumnami `id`, `nazwa`, `adres`.

```
Python
import sqlite3
import pandas as pd
from geopy.geocoders import Nominatim
from geopy.extra.rate_limiter import RateLimiter
import folium

# Połączenie z bazą SQLite
conn = sqlite3.connect("punkty.db")

# Zapytaj o cokolwiek
```

```
Python
conn = sqlite3.connect("punkty.db")

# Pobranie danych
df = pd.read_sql_query("""
SELECT id, nazwa, adres
FROM punkty_dostaw
""", conn)

conn.close()

# Geocoder OpenStreetMap (Nominatim)
geolocator = Nominatim(user_agent="moja_aplikacja")
geocode = RateLimiter(geolocator.geocode, min_delay_seconds=1)

# Zamiana adresów na współrzędne
df["location"] = df["adres"].apply(geocode)

df["lat"] = df["location"].apply(
    lambda loc: loc.latitude if loc else None
)

df["lon"] = df["location"].apply(
    lambda loc: loc.longitude if loc else None
)

# Usunięcie rekordów bez współrzędnych
```

Python

```

# ...
popup=row["adres"]
).add_to(m)

m.save("mapa.html")

```

**Duża liczba punktów**

Jeżeli masz setki lub tysiące punktów dostaw, zamiast zwykłych markerów warto użyć grupowania:

Python

```

from folium.plugins import MarkerCluster

cluster = MarkerCluster().add_to(m)

for _, row in df.iterrows():
    folium.Marker(
        [row["lat"], row["lon"]],
        popup=row["adres"]
    ).add_to(cluster)

```

To znacznie przyspiesza działanie mapy i poprawia czułość. Jeśli pokażesz strukturę swojej tabeli SQLite (CREATE TABLE ... lub wynik PRAGMA table\_info(nazwa\_tabeli)), mogę od razu dopasować gotowy kod do Twojej bazy.

+ Zapytaj o cokolwiek
Głos

ChatGPT to AI może się mylić

- pomoc przy implementacji wybranych funkcji aplikacji
- diagnozowanie i naprawianie błędów pojawiających się podczas pisania kodu

co może sprawić że niektóre poprawnie zapisane adresy nie pojawiają się na wygenerowanej mapie?

Jest kilka typowych powodów, dla których część adresów nie pojawia się na mapie, mimo że są poprawnie zapisane w SQLite.

**1. Geopy nie znajduje lokalizacji**

Najczęstsza przyczyna. Adres jest poprawny z punktu widzenia użytkownika, ale geokoder (np. usługa Nominatim OpenStreetMap) nie potrafi go jednoznacznie zinterpretować.

Przykład:

Python

```

location = geolocator.geocode(adres)

if location is None:
    print(f"Nie znaleziono: {adres}")

```

Warto logować wszystkie adresy, dla których `location is None`.

+ Zapytaj o cokolwiek
Głos

ChatGPT to AI może się mylić

**2. Niejednoznaczny format adresu**

Geokodery lepiej radzą sobie z pełnymi adresami:

✗

Lipowa 5

✓

Lipowa 5, Gdańsk, Polska

✓

ul. Lipowa 5, 80-123 Gdańsk, Polska

Brak miasta, kodu pocztowego lub kraju może powodować błędne wyniki.

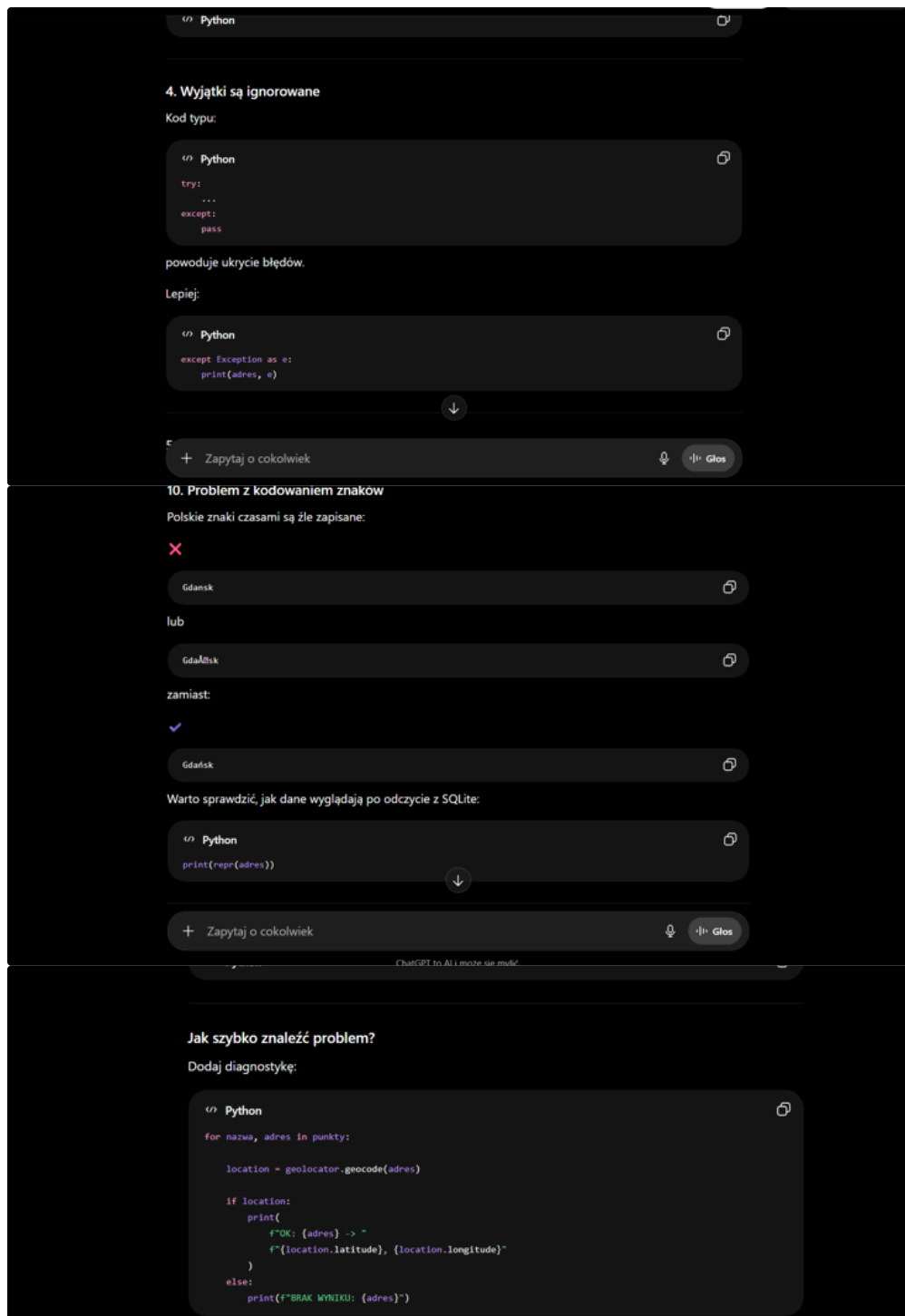
**3. Limity i blokady Nominatim**

Serwer Nominatim ma ograniczenia liczby zapytań.

Jeżeli wykonujesz wiele geokodowań:

+ Zapytaj o cokolwiek
Głos

ChatGPT to AI może się mylić



- pomoc przy przygotowaniu dokumentacji użytkownika
- pomoc przy aktualizacji planu projektu

### 3. Zakres pracy wykonanej samodzielnie

Samodzielnie wykonano:

- projekt koncepcji aplikacji,
- określenie celu i funkcjonalności programu,
- podejmowanie decyzji dotyczących wyglądu i działania interfejsu,

- implementację oraz wprowadzanie zmian do kodu źródłowego,
- testowanie działania programu,
- poprawianie błędów i weryfikację proponowanych rozwiązań,
- przygotowanie diagramów,
- konfigurację repozytorium GitHub,
- sprawdzenie poprawności działania wersji EXE,
- podejmowanie decyzji dotyczących dalszego rozwoju projektu.

#### **4. Ograniczenia wykorzystania AI**

Propozycje generowane przez sztuczną inteligencję wymagały każdorazowej analizy i dostosowania do potrzeb projektu. Nie wszystkie rozwiązania działały dlatego konieczne było samodzielne testowanie programu, poprawianie błędów oraz dostosowywanie kodu do aktualnego stanu aplikacji.

#### **5. Podsumowanie**

Sztuczna inteligencja pełniła rolę narzędzia wspomagającego proces tworzenia projektu i nie zastępowała samodzielnej pracy. Ostateczny kształt aplikacji, decyzje projektowe, testowanie oraz integracja wszystkich elementów zostały wykonane samodzielnie. Wykorzystanie AI pozwoliło usprawnić pracę oraz lepiej zrozumieć niektóre zagadnienia związane z programowaniem i projektowaniem aplikacji.